LAPORAN TUGAS BESAR

LAB. SENSOR DAN TRANDUSER

ALAT PEMADAM KEBAKARAN DAN SISTEM SONAR BERBASIS IOT

DISUSUN OLEH :

1.	BARRON ASWIN SAFFIR	EK-(2C)	(3.32.24.2.04)
2.	HEMA TATA NUGRAHA	EK-(2C)	(3.32.24.2.13)
3.	ZULFI AZMI	EK-(2C)	(3.32.24.2.25)

Dosen Pengampu : Wahyu Sulistiyo, S.T., M.Kom.

PROGRAM STUDI D3 TEKNIK ELEKTRONIKA

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI SEMARANG

TAHUN AJARAN 2025

BIODATA PRAKTIKAN

FOTO	NAMA
	BARRON ASWIN SAFFIR
	HEMA TATA NUGRAHA
	ZULFI AZMI

DAFTAR ISI

BIODATA PRAKTIKAN i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan 1
1.4 Manfaat 1
BAB II LANDASAN TEORI 2
2.2 Dasar Teori 2
2.3 Alat Dan Bahan Serta Wiring 3
2.4 Gambar Rangkaian 4
2.5 Pemograman 5
BAB III HASIL DAN PEMBAHASAN 19
3.1 Hasil Perancangan Sistem 19
3.2 Hasil Pengujian Setiap Komponen 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

a. Mendeteksi api secara otomatis.
b. Mengaktifkan buzzer sebagai alarm ketika ada api.
c. Menyalakan pompa air untuk memadamkan api.
d. Mengarahkan servor agar air menuju sumber api.

1.2 Rumusan Masalah

1. Bagaimana sistem dapat mendeteksi keberadaan api secara otomatis?
2. Bagaimana cara menentukan arah sumber api menggunakan sensor ultrasonik?
3. Bagaimana menggerakkan servo agar dapat mengikuti posisi api?
4. Bagaimana mengaktifkan pompa air secara otomatis saat api terdeteksi?
5. Bagaimana menampilkan status sistem pada LCD dan mengirimkan data ke Blynk?

1.3 Tujuan

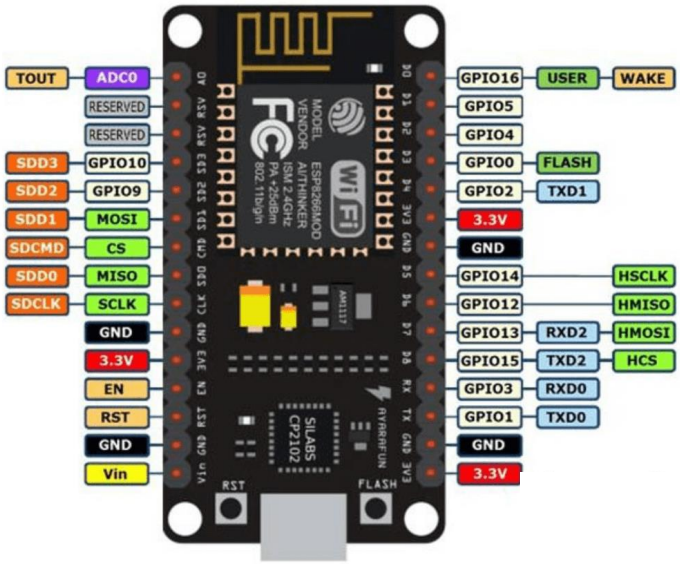
1. Membangun sistem pemadam api otomatis berbasis IoT.
2. Mengembangkan alat yang dapat mendeteksi api dan arah objek menggunakan sensor flame & ultrasonic.
3. Mengarahkan nozzle menggunakan servo secara otomatis menuju titik api.
4. Mengaktifkan pompa air secara otomatis untuk proses pemadaman.
5. Menampilkan informasi sistem pada LCD dan aplikasi Blynk.

1.4 Manfaat

1. Meningkatkan keamanan ruangan dengan deteksi api dini.
2. Meminimalkan kerugian akibat kebakaran dengan respon otomatis.
3. Memberikan sistem monitoring real-time melalui IoT.
4. Meningkatkan pemahaman mahasiswa terhadap sensor, aktuator, dan IoT.

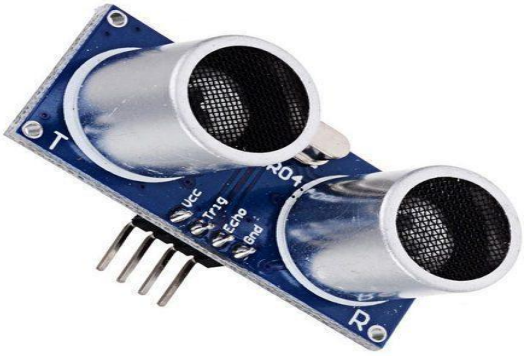
BAB II
LANDASAN TEORI

2.2 Dasar Teori



Gambar 1.Modul ESP8266

Sistem pemadam api otomatis pada proyek ini menggunakan modul ESP-N (ESP8266/ESP32) sebagai pusat pengendali utama yang berfungsi memproses seluruh data dari sensor dan mengatur aktuator. Modul ini merupakan mikrokontroler modern yang dilengkapi kemampuan komputasi serta konektivitas nirkabel, sehingga mampu mengelola sistem berbasis sensor secara cepat dan efektif. Deteksi awal kebakaran dilakukan menggunakan sensor api (flame sensor) yang bekerja berdasarkan penangkapan cahaya inframerah (IR) yang dipancarkan oleh nyala api. Intensitas inframerah yang tinggi mengindikasikan keberadaan api sehingga modul ESP-N mendapatkan sinyal untuk mengaktifkan sistem pemadam.



Gambar 2. Sensor UltraSonic hc-sr04

Gambar 3.LCD 16x2 i2c

Untuk menentukan posisi atau jarak objek di depan sistem, digunakan sensor ultrasonik, yaitu sensor yang memancarkan gelombang suara berfrekuensi tinggi lalu mengukur waktu

pantul dari objek tersebut. Informasi jarak ini digunakan untuk mengatur arah servo sehingga nozzle air dapat diarahkan dengan tepat ke titik api. Servo motor sendiri merupakan aktuator yang mampu bergerak pada sudut tertentu dengan presisi melalui sinyal PWM dari mikrokontroler, memungkinkan sistem melakukan penguncian arah pemadaman secara otomatis.

Gambar 4.Servo SG90

Gambar 5. Pompa Air

Ketika kebakaran terdeteksi, pompa air diaktifkan melalui modul driver atau relay untuk menyemprotkan air. Agar sistem dapat mengetahui apakah air benar-benar mengalir, digunakan flow sensor YF-S201 yang bekerja dengan turbin internal. Putaran turbin menghasilkan pulsa yang dihitung oleh mikrokontroler untuk mengetahui laju aliran air. Dengan demikian, sistem dapat memverifikasi keluarnya air secara real-time.

Seluruh status, seperti kondisi sensor api, nilai jarak ultrasonik, status pompa, arah servo, serta laju aliran air, ditampilkan melalui LCD (Liquid Crystal Display) sehingga pengguna dapat memonitor sistem secara langsung. Integrasi seluruh komponen ini menghasilkan sistem pemadam api otomatis yang mampu mendeteksi, mengarahkan, memverifikasi, serta menampilkan kondisi pemadaman secara cepat dan akurat. Modul ESP-N berperan penting sebagai otak yang menyatukan seluruh fungsi sensor, aktuator, dan tampilan sehingga sistem dapat bekerja secara mandiri dan responsif.

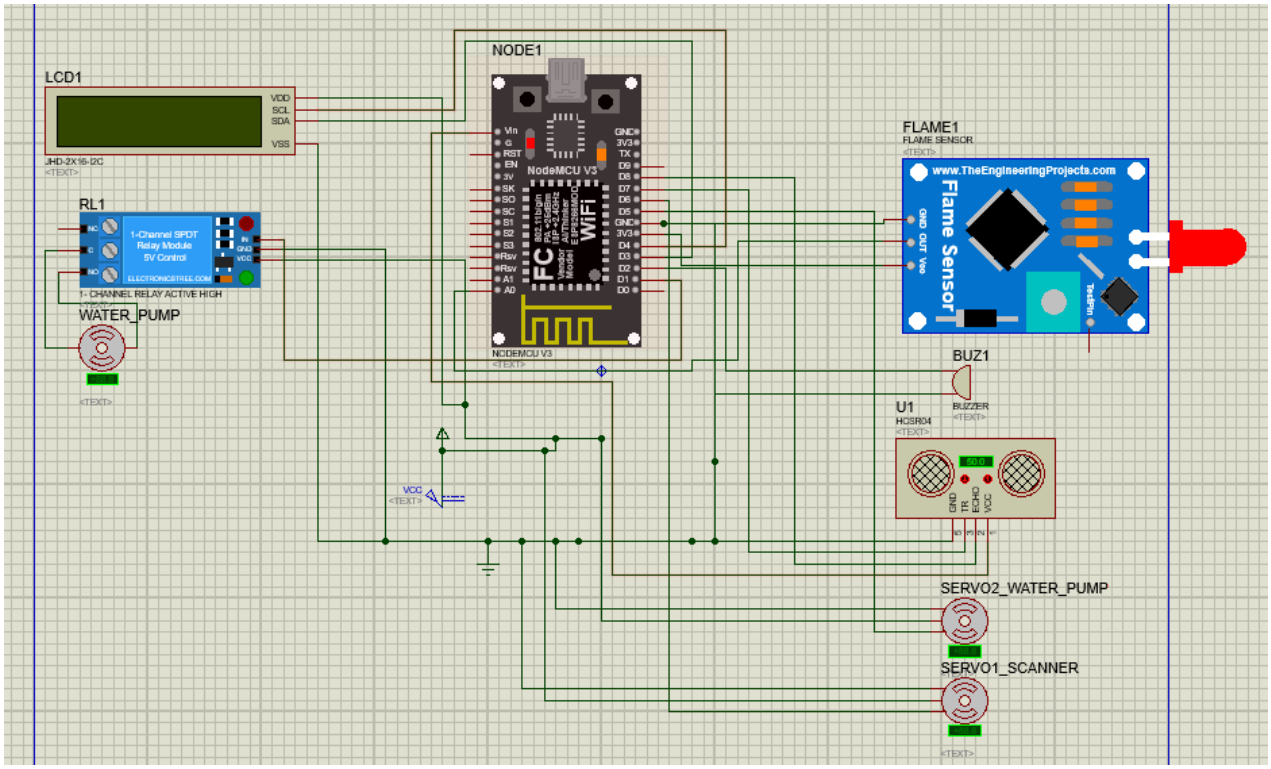
2.3 Alat Dan Bahan Serta Wiring

1. Tabel Alat Bahan dan Wiringnya

Komponen	Pin Komponen	Hubungkan ke
KY-026 Flame Sensor	VCC	3.3V (NodeMCU)
	GND	GND
	AOUT	A0
HC-SR04 Ultrasonic	VCC	5V (powerbank/VIN)

	GND	GND
	TRIG	D7
	ECHO	D8 (melalui divider 10k → 20k)
Servo SG90 #1 (Scan)	VCC	5V powerbank
	GND	GND
	Signal	D5
Servo SG90 #2 (Pump)	VCC	5V powerbank
	GND	GND
	Signal	D6
LCD 16x2 I2C (0x27)	VCC	5V
	GND	GND
	SDA	D3

2.4 Gambar Rangkaian



Gambar 6. Gambar Rangkaian

2.5 Pemograman

```
// PROGRAM ASLI: Fire detector + pump control + Blynk + LCD
// Versi ini MENAMBAHKAN KOMENTAR untuk memudahkan pembacaan
dan pemikiran.
// LOGIKA ASLI TIDAK DIUBAH SEDIKITPUN: hanya komentar
tambahan.

#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <EEPROM.h>   // EEPROM untuk menyimpan kalibrasi

// ============ BLYNK ============
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

char blynk_auth[] = "tcL4LJ0ib78JxoiIdOpu_5cc_BgEMjiy";
char wifi_ssid[]  = "HiFi-13897";
char wifi_pass[]  = "pssslemanale";

BlynkTimer blynkTimer;

// ================== Servo ==================
// Dua servo: satu untuk scan (servoScan) dan satu untuk
mengarahkan pompa (servoPump)
Servo servoScan;
Servo servoPump;

// ====== RANGE GERAK SERVO SCAN ======
// Batas mekanik yang digunakan agar servo tidak mentok
const int SCAN_MIN   = 15;
const int SCAN_MAX   = 165;
const int SCAN_DELAY = 30; // delay per langkah servo scan
(ms)

// sudut sekarang & arah gerak
int scanAngle = SCAN_MIN;
int scanDir   = 1; // 1 = naik, -1 = turun

// ====== SERVO POMPA ======
const int PUMP_CENTER = 90; // posisi tengah pompa
bool pumpTracking = false;  // (tidak dipakai intensif, status
bantu)
// track sudut aktual servo pump (nilai terakhir yang ditulis
ke servoPump)
int currentPumpAngle = PUMP_CENTER;

// ================== Konfigurasi pin ==================
const int PIN_BUZZER = D2; // buzzer/alarma
```

```
const int PIN_RELAY  = D1; // relay kontrol pompa

const int PIN_TRIG   = D7; // HC-SR04 trigger pin
const int PIN_ECHO   = D8; // HC-SR04 echo pin

const int PIN_FLAME  = A0; // Masukan analog dari sensor KY-
026

// I2C LCD pins
const int PIN_I2C_SDA = D3;
const int PIN_I2C_SCL = D4;

// ================== Batas logika ==================
const int   BATAS_API   = 300;   // (tidak digunakan langsung,
apiAda ditentukan oleh logika KY-026)
const float BATAS_JARAK = 10.0;

// ================== Filter params ==================
#define MEDIAN_WINDOW 5
#define EMA_ALPHA     0.25f

#define KALMAN_Q 0.5f
#define KALMAN_R 10.0f

// ================== Kalibrasi HC-SR04 ==================
const unsigned long HC_CAL_TIME_MS  = 10000UL;
const unsigned long HC_ECHO_TIMEOUT = 30000UL;

// hasil regresi awal
float CAL_A = 0.9889f;
float CAL_B = 0.0f;

// ================== EEPROM layout ==================
const int EEPROM_SIZE = 64;
const uint32_t EEPROM_MAGIC = 0xA5A5F0F0UL;
const int ADDR_MAGIC = 0;
const int ADDR_CALA  = ADDR_MAGIC + 4;
const int ADDR_CALB  = ADDR_CALA + 4;
const int ADDR_LEVEL = ADDR_CALB + 4; // alamat untuk
menyimpan level sensitivitas (0..9)

// ================== Flame (KY-026) filter globals
==================
// Kalman untuk KY-026 (variabel terpisah agar tidak
mengganggu kalmanFlame struct yg ada)
float ky_x_est = 0.0;
float ky_P_est = 1.0;
```

```
float ky_Q = 1.0;      // noise proses (akan dikontrol oleh
level)
float ky_R = 6.0;      // noise sensor (akan dikontrol oleh
level)

// sliding window buffer (window ~1 detik, dihitung dari
SAMPLE_INTERVAL_MS)
const int KY_MAX_WINDOW = 50; // batas maksimum
float ky_buffer[KY_MAX_WINDOW];
int ky_indexBuf = 0;
bool ky_bufferFull = false;
int ky_windowSize = 1; // akan di-set di setup sesuai
SAMPLE_INTERVAL_MS

unsigned long lastSample = 0; // (dipakai juga di kode utama
untuk sampling)

// ---------- Simple Kalman (tidak dipakai untuk KY-026) -----
-----
struct SimpleKalman {
  float q, r, x, p, k;

  void init(float q_, float r_, float initial) {
    q = q_;
    r = r_;
    x = initial;
    p = 1.0f;
    k = 0.0f;
  }

  float update(float measurement) {
    p = p + q;
    k = p / (p + r);
    x = x + k * (measurement - x);
    p = (1.0f - k) * p;
    return x;
  }
} kalmanFlame;

// ================== Runtime sampling & thresholds (now
variables) ==================
// GANTI dari #define menjadi variabel agar bisa diubah oleh
level
unsigned long SAMPLE_INTERVAL_MS = 200UL;

// Thresholds yang dulu konstanta — sekarang variabel
dikendalikan oleh level
float MIN_MEAN = 120.0f;
```

```
float MIN_VAR  = 6.0f;
float MIN_REL  = 0.025f;

// ================== LCD I2C ==================
LiquidCrystal_I2C lcd(0x27, 16, 2);

void initLCD() {
  Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
  lcd.init();
  lcd.backlight();

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("FIRE: NONE      ");
  lcd.setCursor(0, 1);
  lcd.print("SCAN:000 DIST:0 ");
}

// ================== Blynk control ==================
bool modeAuto = true; // true = automatic mode, false = manual

int manualScanAngle = SCAN_MIN; // nilai sudut scan jika
manual
int manualPumpAngle = PUMP_CENTER; // nilai sudut pump jika
manual
bool manualPumpRelay = false; // nilai relay jika manual

BLYNK_WRITE(V7) {
  int v = param.asInt();
  modeAuto = (v == 1);

  if (!modeAuto) {
    // saat masuk manual, terapkan nilai manual yang diset di
Blynk
    scanAngle = constrain(manualScanAngle, SCAN_MIN,
SCAN_MAX);
    servoScan.write(scanAngle);

    // set servo pump pada posisi manual
    servoPump.write(constrain(manualPumpAngle, SCAN_MIN,
SCAN_MAX));
    currentPumpAngle = constrain(manualPumpAngle, SCAN_MIN,
SCAN_MAX);

    // relay juga dikendalikan langsung saat manual
    if (manualPumpRelay) digitalWrite(PIN_RELAY, LOW);
    else                 digitalWrite(PIN_RELAY, HIGH);
  } else {
```

```
    // mode auto: biarkan loop() yang mengendalikan
servoPump/relay berdasarkan api
  }

  // update cepat status ke Blynk setelah mode berubah
  Blynk.virtualWrite(V1, digitalRead(PIN_RELAY) == LOW ? 1 :
0);
  Blynk.virtualWrite(V4, currentPumpAngle);
}

BLYNK_WRITE(V0) {
  manualScanAngle = constrain(param.asInt(), SCAN_MIN,
SCAN_MAX);
  if (!modeAuto) {
    // saat manual, ubah sudut scan langsung
    scanAngle = manualScanAngle;
    servoScan.write(scanAngle);
  }
}

BLYNK_WRITE(V4) {
  manualPumpAngle = constrain(param.asInt(), SCAN_MIN,
SCAN_MAX);
  if (!modeAuto) {
    // saat manual, set sudut pompa ke nilai dari Blynk
    servoPump.write(manualPumpAngle);
    currentPumpAngle = manualPumpAngle;
  }
}

BLYNK_WRITE(V1) {
  manualPumpRelay = (param.asInt() != 0);
  if (!modeAuto) {
    // saat mode manual, relay & buzzer dikontrol langsung
oleh slider V1
    if (manualPumpRelay) {
      digitalWrite(PIN_RELAY, LOW);
      digitalWrite(PIN_BUZZER, HIGH);
    } else {
      digitalWrite(PIN_RELAY, HIGH);
      digitalWrite(PIN_BUZZER, LOW);
    }
  }
}

// ================== KY-026 Kalman update ==================
float ky_kalmanUpdate(float z) {
  ky_P_est = ky_P_est + ky_Q;                // PREDICT
```

```
  float K = ky_P_est / (ky_P_est + ky_R);    // GAIN
  ky_x_est = ky_x_est + K * (z - ky_x_est);  // UPDATE
  ky_P_est = (1.0 - K) * ky_P_est;           // UPDATE
COVARIANCE
  return ky_x_est;
}

// ================== Sensitivity level storage & control
==================
uint8_t sensitivityLevel = 7; // default setara setting Anda
sekarang (0..9)

// simpan level ke EEPROM (1 byte)
void saveLevelToEEPROM(uint8_t lvl) {
  EEPROM.write(ADDR_LEVEL, lvl);
  EEPROM.commit();
}

// baca level dari EEPROM (fallback ke 7 jika invalid)
uint8_t readLevelFromEEPROM() {
  uint8_t v = EEPROM.read(ADDR_LEVEL);
  if (v > 9) return 7;
  return v;
}

// fungsi untuk set parameter sesuai level 0..9
#include <math.h>
void setSensitivityLevel(uint8_t lvl) {
  lvl = constrain(lvl, 0, 9);
  sensitivityLevel = lvl;

  switch (lvl) {
    case 0:
      SAMPLE_INTERVAL_MS = 300UL;
      ky_Q = 0.3f; ky_R = 35.0f;
      MIN_MEAN = 170.0f; MIN_VAR = 30.0f; MIN_REL = 0.14f;
      break;

    case 1:
      SAMPLE_INTERVAL_MS = 260UL;
      ky_Q = 0.4f; ky_R = 30.0f;
      MIN_MEAN = 155.0f; MIN_VAR = 26.0f; MIN_REL = 0.12f;
      break;

    case 2:
      SAMPLE_INTERVAL_MS = 230UL;
      ky_Q = 0.5f; ky_R = 24.0f;
      MIN_MEAN = 145.0f; MIN_VAR = 22.0f; MIN_REL = 0.10f;
```

```
      break;

    case 3:
      SAMPLE_INTERVAL_MS = 200UL;
      ky_Q = 0.6f; ky_R = 20.0f;
      MIN_MEAN = 135.0f; MIN_VAR = 18.0f; MIN_REL = 0.08f;
      break;

    case 4:
      SAMPLE_INTERVAL_MS = 180UL;
      ky_Q = 0.7f; ky_R = 16.0f;
      MIN_MEAN = 130.0f; MIN_VAR = 15.0f; MIN_REL = 0.065f;
      break;

    case 5:
      SAMPLE_INTERVAL_MS = 170UL;
      ky_Q = 0.8f; ky_R = 14.0f;
      MIN_MEAN = 125.0f; MIN_VAR = 13.0f; MIN_REL = 0.055f;
      break;

    case 6:
      SAMPLE_INTERVAL_MS = 160UL;
      ky_Q = 0.9f; ky_R = 12.0f;
      MIN_MEAN = 122.0f; MIN_VAR = 11.0f; MIN_REL = 0.045f;
      break;

    case 7:
      SAMPLE_INTERVAL_MS = 150UL;
      ky_Q = 1.0f; ky_R = 10.0f;
      MIN_MEAN = 120.0f; MIN_VAR = 9.0f; MIN_REL = 0.035f;
      break;

    case 8:
      SAMPLE_INTERVAL_MS = 140UL;
      ky_Q = 1.2f; ky_R = 9.0f;
      MIN_MEAN = 118.0f; MIN_VAR = 8.0f; MIN_REL = 0.030f;
      break;

    case 9:
      SAMPLE_INTERVAL_MS = 130UL;
      ky_Q = 1.4f; ky_R = 8.0f;
      MIN_MEAN = 115.0f; MIN_VAR = 7.0f; MIN_REL = 0.025f;
      break;
  }
}



// Blynk callback untuk level (V8) — auto-apply & auto-save
```

```
BLYNK_WRITE(V8) {
  uint8_t lvl = (uint8_t)constrain(param.asInt(), 0, 9);
  if (lvl != sensitivityLevel) {
    setSensitivityLevel(lvl);
    saveLevelToEEPROM(lvl);
  }
}

// ================== HC-SR04 ==================
float bacaJarakCM_raw() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  long durasi = pulseIn(PIN_ECHO, HIGH, HC_ECHO_TIMEOUT);
  if (durasi == 0) return -1.0;

  return (float)durasi * 0.034f / 2.0f;
}

float bacaJarakCM() {
  float d_raw = bacaJarakCM_raw();
  if (d_raw < 0) return -1.0f;
  return CAL_A * d_raw + CAL_B;
}

// ================== EEPROM helper ==================
void saveFloatToEEPROM(int addr, float value) {
  union { float f; uint8_t b[4]; } u;
  u.f = value;
  for (int i = 0; i < 4; i++) EEPROM.write(addr + i, u.b[i]);
}

float readFloatFromEEPROM(int addr) {
  union { float f; uint8_t b[4]; } u;
  for (int i = 0; i < 4; i++) u.b[i] = EEPROM.read(addr + i);
  return u.f;
}

void saveCalibrationToEEPROM(float a, float b) {
  EEPROM.write(ADDR_MAGIC + 0, (uint8_t)((EEPROM_MAGIC >> 0) &
0xFF));
  EEPROM.write(ADDR_MAGIC + 1, (uint8_t)((EEPROM_MAGIC >> 8) &
0xFF));
  EEPROM.write(ADDR_MAGIC + 2, (uint8_t)((EEPROM_MAGIC >> 16)
& 0xFF));
```

```
  EEPROM.write(ADDR_MAGIC + 3, (uint8_t)((EEPROM_MAGIC >> 24)
& 0xFF));
  saveFloatToEEPROM(ADDR_CALA, a);
  saveFloatToEEPROM(ADDR_CALB, b);
  EEPROM.commit();
}

bool loadCalibrationFromEEPROM(float &a_out, float &b_out) {
  uint32_t m = 0;
  m |= ((uint32_t)EEPROM.read(ADDR_MAGIC + 0)) << 0;
  m |= ((uint32_t)EEPROM.read(ADDR_MAGIC + 1)) << 8;
  m |= ((uint32_t)EEPROM.read(ADDR_MAGIC + 2)) << 16;
  m |= ((uint32_t)EEPROM.read(ADDR_MAGIC + 3)) << 24;

  if (m != EEPROM_MAGIC) return false;

  a_out = readFloatFromEEPROM(ADDR_CALA);
  b_out = readFloatFromEEPROM(ADDR_CALB);
  return true;
}

// ================== SETUP ==================
void setup() {
  Serial.begin(9600);
  EEPROM.begin(EEPROM_SIZE);

  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);

  digitalWrite(PIN_BUZZER, LOW);
  digitalWrite(PIN_RELAY, HIGH);

  // init ky buffer dengan NAN
  for (int i = 0; i < KY_MAX_WINDOW; i++) ky_buffer[i] = NAN;

  // init kalmanFlame original (keamanan, tidak dipakai untuk
KY-026)
  kalmanFlame.init(KALMAN_Q, KALMAN_R, 500.0f);

  // init KY-026 kalman (ambil nilai awal dari analog)
  int first = analogRead(PIN_FLAME);
  ky_x_est = (float)first;
  ky_P_est = 1.0f;

  // ===== load sensitivity level from EEPROM and apply BEFORE
computing window =====
```

```
  sensitivityLevel = readLevelFromEEPROM();
  setSensitivityLevel(sensitivityLevel);

  // set dynamic window size (≈1 detik) based on
SAMPLE_INTERVAL_MS
  int w = (int)(1000UL / SAMPLE_INTERVAL_MS);
  if (w < 1) w = 1;
  if (w > KY_MAX_WINDOW) w = KY_MAX_WINDOW;
  ky_windowSize = w;

  ky_indexBuf = 0;
  ky_bufferFull = false;

  servoScan.attach(D5);
  servoPump.attach(D6);

  servoScan.write(SCAN_MIN);
  servoPump.write(PUMP_CENTER);
  currentPumpAngle = PUMP_CENTER;

  initLCD();

  float a_e, b_e;
  if (loadCalibrationFromEEPROM(a_e, b_e)) {
    CAL_A = a_e;
    CAL_B = b_e;
  }

  // Start Blynk
  Blynk.begin(blynk_auth, wifi_ssid, wifi_pass);

  // sync slider with current level
  Blynk.virtualWrite(V8, sensitivityLevel);

  manualScanAngle = scanAngle;
  manualPumpAngle = PUMP_CENTER;

  lastSample = millis();
}

// ================== LOOP ==================
void loop() {
  // ====== 1. SERVO SCAN ======
  servoScan.write(scanAngle);
  delay(SCAN_DELAY);

  // ====== 2. BACA JARAK ======
  float jarakRadar = bacaJarakCM();
```

```
  int jarakRadarInt = (jarakRadar < 0) ? 0 : (int)jarakRadar;

  // ====== 3. UPDATE SCAN ======
  if (modeAuto) {
    scanAngle += scanDir;
    if (scanAngle >= SCAN_MAX) scanDir = -1;
    if (scanAngle <= SCAN_MIN) scanDir = 1;
  } else {
    scanAngle = manualScanAngle;
  }

  // ====== 4. SENSOR API (KY-026) ======
  unsigned long now = millis();
  if (now - lastSample >= SAMPLE_INTERVAL_MS) {
    lastSample = now;

    // --- baca raw ---
    int rawADC = analogRead(PIN_FLAME);

    // --- kalman ---
    float filtered = ky_kalmanUpdate((float)rawADC);

    // --- simpan ke buffer sliding window (~1 detik) ---
    ky_buffer[ky_indexBuf] = filtered;
    ky_indexBuf++;
    if (ky_indexBuf >= ky_windowSize) {
      ky_indexBuf = 0;
      ky_bufferFull = true;
    }

    bool apiAda = false; // default

    if (ky_bufferFull) {
      float minVal = 1023.0f;
      float maxVal = 0.0f;
      float sumVal = 0.0f;

      for (int i = 0; i < ky_windowSize; i++) {
        float v = ky_buffer[i];
        if (v < minVal) minVal = v;
        if (v > maxVal) maxVal = v;
        sumVal += v;
      }

      float mean = sumVal / ky_windowSize;
      float variability = maxVal - minVal;
      float relVar = (mean > 0.0f) ? (variability / mean) :
0.0f;
```

```
      // THRESHOLD DETEKSI API (menggunakan variabel yang di-
set oleh level)
      if (mean > MIN_MEAN && variability > MIN_VAR && relVar >
MIN_REL) {
        apiAda = true;
      } else {
        apiAda = false;
      }
    }

    // ====== 6. POMPA (logika utama, gunakan apiAda dari KY-
026) ======
    if (modeAuto) {
      if (apiAda) {
        // pump mengikuti scanAngle (tapi kita simpan nilai
aktual di currentPumpAngle)
        servoPump.write(scanAngle);
        currentPumpAngle = constrain(scanAngle, SCAN_MIN,
SCAN_MAX);

        digitalWrite(PIN_RELAY, LOW);
        digitalWrite(PIN_BUZZER, HIGH);
      } else {
        digitalWrite(PIN_RELAY, HIGH);
        digitalWrite(PIN_BUZZER, LOW);
      }
    } else {
      servoPump.write(manualPumpAngle);
      currentPumpAngle = constrain(manualPumpAngle, SCAN_MIN,
SCAN_MAX);

      if (manualPumpRelay || apiAda) {
        digitalWrite(PIN_RELAY, LOW);
        digitalWrite(PIN_BUZZER, HIGH);
      } else {
        digitalWrite(PIN_RELAY, HIGH);
        digitalWrite(PIN_BUZZER, LOW);
      }
    }

    // ====== 7. LCD (UPDATE) ======
    lcd.setCursor(0, 0);

    // BARIS 1 LCD
    lcd.setCursor(0, 0);

    // Fire0-9: Yes/No (MODE)
```

```
    lcd.print("Fire");
    lcd.print(sensitivityLevel);   // 0-9 TANPA offset
    lcd.print(":");

    if (apiAda) lcd.print("Yes");
    else        lcd.print("No ");

    if (modeAuto) lcd.print(" AUTO  ");
    else          lcd.print(" MANUAL");

    // ====== BARIS KE-2 (TETAP) ======
    int angleDisp = constrain(scanAngle, 0, 180);

    char line2[17];
    if (jarakRadarInt > 99) {
      snprintf(line2, sizeof(line2), "SCAN:%03d DIST:99+",
angleDisp);
    } else {
      snprintf(line2, sizeof(line2), "SCAN:%03d DIST:%02d",
angleDisp, jarakRadarInt);
    }

    lcd.setCursor(0, 1);
    lcd.print(line2);

    // ====== 8. BLYNK SEND ======
    // Scan angle
    Blynk.virtualWrite(V0, angleDisp);

    // Status relay/pump nyata
    int relayState = digitalRead(PIN_RELAY); // LOW atau HIGH
    int pumpOnForBlynk = (relayState == LOW) ? 1 : 0;
    Blynk.virtualWrite(V1, pumpOnForBlynk);

    // Status deteksi api
    Blynk.virtualWrite(V2, (int)apiAda);

    // Jarak radar
    Blynk.virtualWrite(V3, jarakRadarInt);

    // Sudut pompa nyata
    Blynk.virtualWrite(V4, currentPumpAngle);
  }

  // ====== 9. SERIAL UNTUK PROCESSING ======
  Serial.print(scanAngle);
  Serial.print(",");
  Serial.print(jarakRadarInt);
```

```
  Serial.print(".");

  Blynk.run();
  blynkTimer.run();
}
```

BAB III
HASIL DAN PEMBAHASAN

3.1 Hasil Perancangan Sistem

Proyek yang dibangun merupakan sistem pemadam kebakaran otomatis berbasis IoT menggunakan mikrokontroler ESP8266. Sistem ini dapat:

1. **Mendeteksi keberadaan api** menggunakan sensor flame KY-026.
2. **Mengukur jarak dan posisi objek** menggunakan sensor ultrasonik HC-SR04.
3. **Menggerakkan servo scan** secara sweeping untuk mencari sumber api.
4. **Mengunci posisi servo pompa** mengikuti arah titik api.
5. **Mengaktifkan pompa air dan buzzer** secara otomatis.
6. **Menampilkan status sistem** melalui LCD I2C 16x2.
7. **Mengirim data real-time** ke aplikasi Blynk (status api, jarak, servo, pompa).

Seluruh proses berjalan secara otomatis dan terintegrasi sehingga alat dapat memadamkan api tanpa campur tangan manusia.

3.2 Hasil Pengujian Setiap Komponen

1. Sensor Flame KY-026

Pengujian dilakukan dengan sumber api kecil (korek api / lilin).
Hasilnya:

- Sensor mampu mendeteksi cahaya IR dari api.
- Output analog berubah signifikan ketika ada nyala api.
- Filter Kalman + membuat deteksi lebih akurat dan mengurangi false trigger dari cahaya lampu/sinar matahari.

Status ditampilkan di LCD:

- "FIRE: NONE" → tidak ada api
- "FIRE: DETECTED" → api terdeteksi

Status juga dikirim ke Blynk (V2).

2. Sensor Ultrasonik HC-SR04

Pengujian dilakukan dengan objek pada jarak 5–99 cm.

Hasil:

- Sensor mampu membaca jarak yang stabil setelah proses kalibrasi (A dan B menggunakan EEPROM).
- Jarak digunakan untuk menampilkan nilai pada LCD.
- Data dikirim ke Blynk pada V3.

Dividing resistor 10 kΩ–20 kΩ berhasil menurunkan sinyal ECHO 5V menjadi aman untuk pin D8 ESP8266 (3.3V).

3. Servo SCAN (D5)

Servo scan akan:

- Bergerak dari sudut 15° sampai 165°
- Kembali lagi dengan pola sweep
- Delay 30 ms setiap perubahan sudut

Fungsi servo scan:

- Menyapu area depan alat
- Menentukan arah sumber api
- Memberikan sudut acuan ke servo pompa

Servo bergerak stabil saat menggunakan suplai 5V dari powerbank.

4. Servo POMPA (D6)

Servo pompa mengikuti arah servo scan **hanya ketika api terdeteksi**.

Mode:

- **Tidak ada api → servo kembali ke tengah (90°)**.
- **Api terdeteksi → servo menyesuaikan sudut scanAngle** sehingga semprotan air mengarah ke titik api.

Status sudut servo dikirim ke Blynk (V4).

5. Relay dan Pompa Air

Relay diatur dengan logika:

- **LOW = Pompa ON**
- **HIGH = Pompa OFF**

Kinerja:

- Saat api terdeteksi → relay ON → pompa menyala → air disemprotkan.
- Buzzer ikut ON sebagai alarm.

Pompa mendapat suplai 5V dari powerbank dan mampu menyemburkan air ke arah servo dengan baik.

6. LCD I2C 16x2

LCD menampilkan 2 baris:

Baris 1:
FIRE: DETECTED atau FIRE: NONE

Baris 2:
SCAN:xxx DIST:yy

- xxx = sudut servo scan
- yy = jarak objek (dibatasi 99 cm)

LCD berfungsi normal setelah inisialisasi pada SDA (D3) dan SCL (D4).

Integrasi Blynk IoT

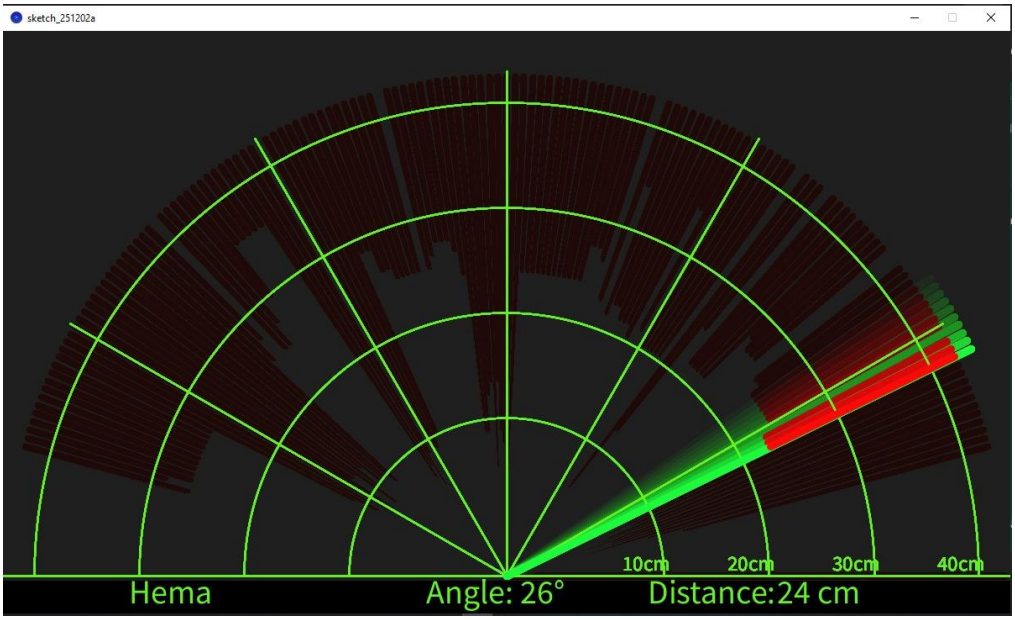
V-pin	Fungsi
V0	Sudut servo scan
V1	Status relay/pompa
V2	Status api
V3	Jarak ultrasonic
V4	Sudut servo pompa
V7	Mode Auto/Manual

Dalam mode manual, pengguna dapat:

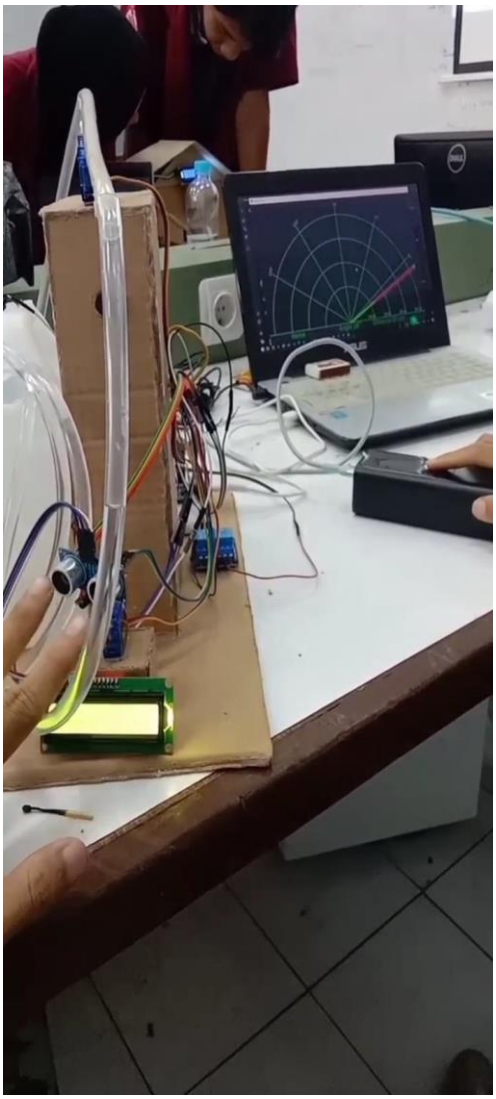
- Mengatur sudut servo scan
- Mengatur servo pompa
- Menghidupkan/mematikan pompa air

Integrasi berjalan baik menggunakan WiFi KONOHA dengan token Blynk.

LAMPIRAN



Gambar 7. Gambar Hasil Sonar Sistem



Gambar 6. Hasil Percobaan Dalam Ruangan

Gambar 6. Hasil Percobaan Luar Ruangan